

MODULE – 01

Design Patterns and Principles.

SOLID Principles and Design Patterns

Introduction

Software design principles help developers create software that is **clean, maintainable, reusable, and scalable**. They reduce complexity and make applications easier to modify and manage.

Two important concepts are:

- **SOLID Principles** – Guidelines for better object-oriented design.
- **Design Patterns** – Reusable solutions for common software design problems.

SOLID Principles

1. Single Responsibility Principle (SRP)

A class should have **only one responsibility** and one reason to change.

Benefits:

- Easy maintenance
- Better testing
- Reduced complexity

2. Open/Closed Principle (OCP)

Software components should be **open for extension but closed for modification**.

New features should be added without changing existing code.

Benefits:

- Improves scalability
- Prevents breaking existing functionality.

3. Liskov Substitution Principle (LSP)

Objects of a child class should be replaceable with objects of the parent class without affecting the application.

Benefits:

- Better inheritance design
- Prevents unexpected behavior

4. Interface Segregation Principle (ISP)

Classes should not depend on unnecessary methods. Interfaces should be small and specific.

Benefits:

- Reduces dependencies
- Improves flexibility

5. Dependency Inversion Principle (DIP)

High-level modules should depend on abstractions instead of specific implementations.

Benefits:

- Loose coupling
- Easier testing
- Flexible design

Design Patterns

Design patterns are reusable approaches to solve common software design problems.

They are divided into three categories:

1. Creational Patterns

Focus on object creation.

Singleton Pattern

Ensures only one instance of a class exists.

Uses: Database connections, configuration management.

Factory Method Pattern

Creates objects without exposing the creation process.

Benefits: Flexible and reduces dependency.

Builder Pattern

Used for creating complex objects step-by-step.

Benefits: Simplifies object creation.

2. Structural Pattern

Focus on organizing classes and objects.

Adapter Pattern

Allows incompatible components to work together.

Decorator Pattern

Adds new features to objects dynamically.

Proxy Pattern

Controls access to another object.

Benefits:

- Better flexibility
- Improved reusability

3. Behavioral Patterns

Focus on communication between objects.

Observer Pattern

Notifies multiple objects when a change occurs.

Example: Notification systems.

Strategy Pattern

Allows changing algorithms dynamically.

Example: Different payment methods.

Command Pattern

Encapsulates requests as objects.

Example: Undo/Redo operations

Architectural Patterns

MVC (Model-View-Controller)

Separates an application into:

- **Model:** Handles data and business logic.
- **View:** Manages user interface.
- **Controller:** Handles user requests and connects Model and View.

Benefits:

- Better organization
- Easy maintenance
- Scalable applications

Dependency Injection (DI)

A technique where dependencies are provided externally instead of being created inside a class.

Benefits:

- Reduces coupling
- Improves flexibility
- Makes testing easier

Advantages of SOLID and Design Patterns

- Improves code quality
- Reduces duplication
- Makes software easier to maintain
- Supports scalability
- Encourages reusable components
- Simplifies future changes

Conclusion

SOLID principles and design patterns help developers build reliable, flexible, and maintainable software. They provide guidelines for writing better code and solving common design challenges efficiently.

```

1 using System;
2 class Logger
3 {
4     private static Logger instance;
5     private Logger()
6     {
7         Console.WriteLine("Logger Instance Created");
8     }
9     public static Logger GetInstance()
10    {
11        if (instance == null)
12        {
13            instance = new Logger();
14        }
15        return instance;
16    }
17    public void Log(string message)
18    {
19        Console.WriteLine("LOG: " + message);

```

```

20    }
21 }
22 class Program
23 {
24     static void Main(string[] args)
25     {
26         Logger logger1 = Logger.GetInstance();
27         logger1.Log("First Message");
28         Logger logger2 = Logger.GetInstance();
29         logger2.Log("Second Message");
30         if (logger1 == logger2)
31         {
32             Console.WriteLine("Only one Logger instance exists.");
33         }
34         else
35         {
36             Console.WriteLine("Multiple Logger instances exist.");

```

```

36             Console.WriteLine("Multiple Logger instances exist.");
37         }
38     }
39 }

```

```
Output
▲ Logger Instance Created
LOG: First Message
LOG: Second Message
Only one Logger instance exists.

=== Code Execution Successful ===
```

Exercise 2: Implementing the Factory Method Pattern

Objective

The objective of this exercise is to understand and implement the **Factory Method Design Pattern**. The Factory Method Pattern is a creational design pattern that provides an interface for creating objects while allowing subclasses to decide the type of object that will be created.

Scenario

A document management system needs to create different types of documents such as **Word, PDF, and Excel documents**. Instead of directly creating objects, the Factory Method Pattern is used to handle object creation efficiently.

This approach improves flexibility and makes the system easier to extend when new document types are added.

Factory Method Pattern

The Factory Method Pattern defines a method for creating objects in a parent class, while allowing child classes to decide which object should be created.

It separates object creation logic from the main application logic.

Implementation Steps

1. Create Document Interface

An interface named `IDocument` is created to define common behavior for all document types.

All document classes implement this interface and provide their own implementation.

2. Create Concrete Document Classes

Different document types are created:

WordDocument

Represents a Word document.

PdfDocument

Represents a PDF document.

ExcelDocument

Represents an Excel document.

Each class implements the document interface and provides a method to create the respective document.

3. Create Abstract Document Factory

An abstract class called `DocumentFactory` is created.

It contains the factory method:

- `CreateDocument()`

This method is responsible for creating document objects.

4. Create Concrete Factory Classes

Separate factory classes are created for each document type:

WordDocumentFactory

Creates Word document objects.

PdfDocumentFactory

Creates PDF document objects.

ExcelDocumentFactory

Creates Excel document objects.

Each factory overrides the `CreateDocument()` method and returns the required document type.

5. Testing the Factory Method

A test class is created to verify the implementation.

The program creates different factories and uses them to generate different documents.

The factory decides which document object should be created without exposing the object creation process.

Advantages of Factory Method Pattern

- Reduces dependency between classes.
- Provides flexibility in object creation.
- Makes code easier to maintain.
- Supports adding new document types without modifying existing code.
- Improves scalability and reusability.

Applications

Factory Method Pattern is commonly used in:

- Document management systems
- Database connection creation
- Payment processing systems
- UI component creation
- File handling applications

Conclusion

The Factory Method Pattern provides an efficient way to create objects by separating object creation from usage. In this exercise, different document types such as Word, PDF, and Excel are created using separate factories. This makes the system flexible, maintainable, and easy to extend with new document formats.


```
1  using System;
2  interface IDocument
3  {
4      void Create();
5  }
6  class WordDocument : IDocument
7  {
8      public void Create()
9      {
10         Console.WriteLine("Word Document Created");
11     }
12 }
13 class PdfDocument : IDocument
14 {
15     public void Create()
16     {
17         Console.WriteLine("PDF Document Created");
18     }
19 }
```

```
20 class ExcelDocument : IDocument
21 {
22     public void Create()
23     {
24         Console.WriteLine("Excel Document Created");
25     }
26 }
27 abstract class DocumentFactory
28 {
29     public abstract IDocument CreateDocument();
30 }
31 class WordDocumentFactory : DocumentFactory
32 {
33     public override IDocument CreateDocument()
34     {
35         return new WordDocument();
36     }
37 }
38 class PdfDocumentFactory : DocumentFactory
```

```

39 {
40     public override IDocument CreateDocument()
41     {
42         return new PdfDocument();
43     }
44 }
45 class ExcelDocumentFactory : DocumentFactory
46 {
47     public override IDocument CreateDocument()
48     {
49         return new ExcelDocument();
50     }
51 }
52 class Program
53 {
54     static void Main(string[] args)
55     {
56         DocumentFactory wordFactory = new WordDocumentFactory();
57         IDocument word = wordFactory.CreateDocument();

```

```

58         word.Create();
59         DocumentFactory pdfFactory = new PdfDocumentFactory();
60         IDocument pdf = pdfFactory.CreateDocument();
61         pdf.Create();
62         DocumentFactory excelFactory = new ExcelDocumentFactory
            ();
63         IDocument excel = excelFactory.CreateDocument();
64         excel.Create();
65     }
66 }

```

Output

```

Word Document Created
PDF Document Created
Excel Document Created

```

QUIZ:

25. What is Template Design Pattern?

- A) It defines the skeleton of an algorithm in a method, deferring some steps to subclasses.
- B) It ensures that a class has only one instance and provides a global point of access to it.
- C) It encapsulates a request as an object, thereby allowing for the parameterization of clients with queues, requests, and operations.
- D) It provides a way to access the elements of an aggregate object sequentially without exposing its underlying representation.

[Click to View Answer and Explanation](#)

Answer:

- A) It defines the skeleton of an algorithm in a method, deferring some steps to subclasses.